# Frequent Itemset Mining on Movie Casts: A Market-Basket Analysis of the IMDB Top 1000 Dataset

Algorithms for Massive Data
Master in Data Science for Economics
Università degli Studi di Milano
Areg Karapetyan 63835A

June 28, 2026

**Abstract**

This report applies market-basket analysis to the problem of finding actors who recur together across movies. Each movie in the IMDB Top 1000 dataset is treated as a transaction (*basket*), and its four credited lead actors are treated as the items inside that basket. We implement the **A-Priori algorithm from scratch** to mine frequent itemsets of co-occurring actors, derive association rules (confidence, lift) from those itemsets, and validate the implementation against the `mlxtend` library. The solution is designed to be data-agnostic: a configurable flag allows the same pipeline to run on the full dataset or on a random subsample, and the support threshold is expressed as a fraction of the data rather than a fixed count, so that it scales automatically with dataset size. On the full dataset of 1000 movies, the algorithm discovers 299 frequent itemsets – including three actor trios that correspond exactly to three well-known movie franchise casts – and all results match the `mlxtend` reference implementation exactly. A dedicated scalability experiment, run across dataset sizes from 100 to 1000 baskets, both confirms the intended adaptive behaviour of the threshold and exposes a concrete corner case in which it can degenerate, which we discuss explicitly.

### Statement of Work

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study

## Contents

# 1 Introduction and Motivation

Market-basket analysis is a classical data-mining technique originally developed to answer questions such as "which products are frequently purchased together?" in retail transaction data. The same technique generalizes to any setting where data can be expressed as a collection of *baskets*, each containing a set of discrete *items*: the goal is to discover **frequent itemsets** – groups of items that co-occur across baskets more often than chance would suggest.

In this project we repurpose this technique for a different domain: movie casts. We treat each movie as a basket and its leading actors as items, and ask *which actors tend to appear together across multiple films*. This is a natural fit for frequent itemset mining: actor co-occurrence is sparse (most pairs of actors never share a film at all), baskets are small and uniform in size (at most four items), and genuinely frequent co-occurring groups are interpretable and verifiable by hand (e.g. a recurring cast in a film franchise).

The central algorithmic challenge – and the actual learning objective of this assignment – is that naively enumerating every possible group of actors does not scale. With on the order of 2,700 distinct actors in this dataset, there are over 3.6 million possible *pairs* alone, and astronomically more triples and quadruples. The **A-Priori algorithm** solves this with a single structural property of support, called *downward closure*: an itemset cannot be frequent unless every one of its subsets is also frequent. This allows the algorithm to build up candidate itemsets level by level, discarding the vast majority of the search space without ever having to count it against the data.

The remainder of this report is organized to mirror how the project was evaluated: Section 2 describes the dataset and which parts of it were used; Section 3 describes how the data was organized into the basket model; Section 4 describes the pre-processing applied; Section 5 describes the algorithms and their implementations; Section 6 addresses scalability explicitly, both by design and through complexity analysis; Section 7 describes the experiments performed and how to reproduce them; Section 8 reports and discusses the experimental results; and Section 9 concludes.

# 2 Dataset

## 2.1 Source

We use the *IMDB Top 1000 Movies & TV Shows* dataset, publicly available on Kaggle (https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows, file `imdb_top_1000.csv`, licensed CC0-1.0). It contains 1000 rows (one per title) and 16 columns: `Poster_Link`, `Series_Title`, `Released_Year`, `Certificate`, `Runtime`, `Genre`, `IMDB_Rating`, `Overview`, `Meta_score`, `Director`, `Star1`, `Star2`, `Star3`, `Star4`, `No_of_Votes`, and `Gross`.

## 2.2 Parts of the dataset considered

This project's question is specifically about actor co-occurrence, so only two parts of the dataset are used:

- `Star1`–`Star4`: the four top-billed actors of each title, as credited on IMDB. These four columns are the entire basis of the analysis – they are converted directly into the items of each basket. They are already fixed by the dataset itself (sourced from IMDB's own billing order); the analysis does not select, rank, or otherwise decide which actors represent a movie.
- `Series_Title`: used only as a human-readable identifier during exploratory data analysis (e.g. to name which specific movies have a data-quality quirk), never as a mining feature.

All other columns (`Genre`, `Director`, `IMDB_Rating`, `Released_Year`, `Runtime`, `Certificate`, `Overview`, `Meta_score`, `No_of_Votes`, `Gross`, `Poster_Link`) are **deliberately excluded**. They describe properties of a movie as a whole rather than its cast, and are not needed to answer the question this project asks; including them would not change the basket model and would only add irrelevant dimensions to the analysis.

# 3 How the Data Was Organized

The dataset is organized into the classical *basket model* used by market-basket analysis:

- Each **row** (movie or TV show) becomes one **basket**.
- Each basket is represented as a Python `set` containing the (non-missing) values of `Star1` through `Star4` for that row.
- The complete dataset becomes a list of $n$ baskets, $\mathcal{B} = \{b_1, \ldots, b_n\}$, with $n = 1000$ on the full dataset.

Using a `set` rather than a `list` for each basket is a deliberate organizational choice with two consequences, both load-bearing for the rest of the pipeline:

1. It automatically de-duplicates repeated names within a single row
2. It provides the hashable, unordered, subset-testable data structure that the A-Priori implementation requires throughout: itemsets themselves are represented as Python `frozenset` objects (an immutable set), which can be used as dictionary keys (to map an itemset to its support count) and as members of other sets (the candidate sets at each level of the algorithm), and which support the $O(1)$-amortized subset test (`candidate.issubset(basket)`) used by the counting routine.

The full pipeline is also parameterized by two configuration flags, `USE_FULL_DATA` and `SAMPLE_SIZE`, that determine *which* rows of the dataset become baskets before the rest of the pipeline runs: either every row (the default), or a reproducible random subsample of `SAMPLE_SIZE` rows (fixed random seed, for replicability). This decision point is placed as early as possible in the pipeline – immediately after cleaning, before basket construction – so that every later step (basket construction, mining, rule generation) operates uniformly regardless of how much data was selected, which is the mechanism that makes the rest of this report's scalability discussion possible.

## 4 Pre-processing

Three pre-processing steps are applied before mining begins:

**Missing/blank-value handling.** Every value in `Star1`–`Star4` is passed through a cleaning function that trims surrounding whitespace and converts missing (`NaN`) or blank-after-trimming values to a sentinel `None`, which is then excluded when the basket is built. On this particular dataset, no missing or blank values were found in any of the four `Star` columns – but the cleaning step is retained regardless, since the pipeline should not silently assume that future or differently-sourced data will be this clean.

**Exact-duplicate collapse.** Four movies list the *same actor's name twice*, verbatim, across their four `Star` columns – for example, *Singin' in the Rain* lists "Gene Kelly" in both `Star1` and `Star2`. Building each basket as a `set` causes this duplicate to collapse automatically, producing a basket of three unique actors instead of four. This is a genuine property of the source data, not a processing artifact: the four affected titles are *Singin' in the Rain* (Gene Kelly), *The General* (Buster Keaton), *What We Do in the Shadows* (Taika Waititi), and *Il Postino* (Massimo Troisi). The resulting basket-size distribution – 996 baskets of size 4 and 4 baskets of size 3, out of 1000 total. Note that this de-duplication only catches *exact* string repeats; near-duplicate spellings of the same real actor (which do not occur in this dataset) would not be merged.

**Optional subsampling.** As described in Section 3, when `USE_FULL_DATA` is set to `False`, a fixed-seed random sample of `SAMPLE_SIZE` rows is drawn *before* basket construction, so that all later steps are agnostic to whether they are operating on the full dataset or a subsample.

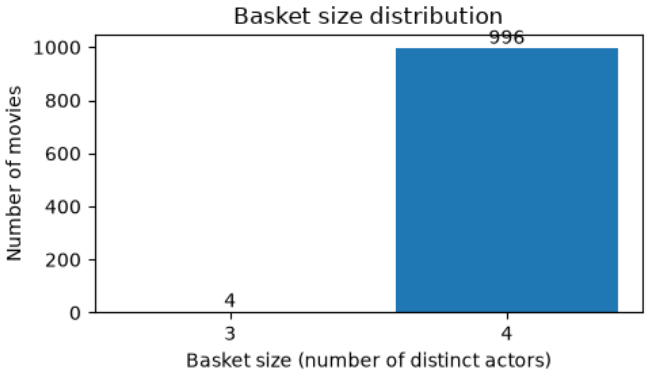


Figure 1: Distribution of basket sizes across all 1000 movies, after pre-processing. The vast majority of baskets contain exactly four actors, as expected; four baskets contain only three due to an exact duplicate actor name within the same row.

As a final pre-mining step, we quantify how sparse actor co-occurrence actually is, since this directly motivates the choice of support threshold. Across the 1000 baskets there are $N = 2{,}709$ distinct actors, giving $\binom{N}{2} = 3{,}667{,}986$ *possible* actor pairs. Of these, only 5,833 pairs are ever actually observed co-starring in at least one movie – a coverage of just 0.159%. The overwhelming majority of all conceivable actor pairs never occur even once in this dataset, which confirms that genuinely repeated co-occurrence (appearing together in several different movies) is a meaningful, non-trivial signal rather than noise.

# 5 Considered Algorithms and Their Implementations

## 5.1 Problem formulation

Let $\mathcal{B} = \{b_1, \ldots, b_n\}$ be the set of baskets, where each $b_i$ is a set of items (actors) drawn from a universe $\mathcal{I}$ of all distinct actors. For an itemset $X \subseteq \mathcal{I}$, define its **support count** as

$$\text{count}(X) = \big|\{b \in \mathcal{B} : X \subseteq b\}\big|,$$

i.e. the number of baskets containing every item in $X$. An itemset $X$ is **frequent** if $\text{count}(X) \geq \tau$ for a chosen minimum support threshold $\tau$. The goal of the mining step is to enumerate all frequent itemsets without exhaustively testing every $X \subseteq \mathcal{I}$, since $|\mathcal{I}| \approx 2{,}709$ makes that infeasible beyond very small itemset sizes.

## 5.2 A-Priori (primary algorithm, implemented from scratch)

A-Priori exploits the following monotonicity property of support: if $X \subseteq Y$, then $\text{count}(Y) \leq \text{count}(X)$, because every basket containing $Y$ must also contain $X$. Equivalently, by contraposition: **if any subset of $X$ is not frequent, then $X$ cannot be frequent either.** This is the *downward-closure* property, and it is the basis for generating only a small set of plausible candidates at each itemset size, instead of every combinatorially possible one.

The algorithm proceeds level by level, where level $k$ corresponds to itemsets of size $k$:

**Algorithm 1** A-Priori (as implemented)

$L_1 \leftarrow$ { frequent itemsets of size 1, found by counting every distinct item directly }
$k \leftarrow 2$
**while** $L_{k-1} \neq \emptyset$ **do**
    $C_k \leftarrow \textsc{Join}(L_{k-1})$ ▷ union pairs of itemsets in $L_{k-1}$ sharing $k-2$ items
    $C_k \leftarrow \textsc{Prune}(C_k, L_{k-1})$ ▷ discard candidates with a non-frequent $(k-1)$-subset
    **if** $C_k = \emptyset$ **then**
        **break**
    **end if**
    count every candidate in $C_k$ against all baskets
    $L_k \leftarrow$ { candidates in $C_k$ with count $\geq \tau$ }
    **if** $L_k = \emptyset$ **then**
        **break**
    **end if**
    $k \leftarrow k + 1$
**end while**
**return** $L_1 \cup L_2 \cup \ldots$

The two non-trivial steps are implemented as follows:

**Join** Candidates of size $k$ are built *only* from pairs of already-frequent itemsets of size $k-1$ whose union has exactly $k$ elements (i.e. they share $k-2$ items). This avoids ever generating

a candidate built from an item or sub-combination that was not already known to be frequent.

**Prune** Even after the join, a candidate $X$ of size $k$ is discarded if *any* of its $(k-1)$-subsets is not already in $L_{k-1}$. This check is performed combinatorially, without touching the basket data at all – it is the source of the algorithm's efficiency, since it eliminates candidates before the (comparatively expensive) step of scanning every basket to count them.

Counting itself is implemented as a single reusable routine that, given a list of candidate itemsets, scans every basket once and increments a counter for each candidate that is a subset of that basket. The same routine is used at every level $k$, including level 1, so the counting logic is identical and independently verifiable at every stage of the algorithm. The loop terminates naturally once either no candidates survive pruning, or no surviving candidates meet the support threshold. Since baskets in this dataset contain at most four items, the loop can never meaningfully proceed past $k = 4$.

### 5.3 Association rule generation

As a second technique applied on top of the frequent itemsets produced by A-Priori, we generate association rules. For every frequent itemset of size $\geq 2$, we consider every way of splitting it into a non-empty antecedent $A$ and consequent $B = X \setminus A$, and compute:

$$\text{confidence}(A \Rightarrow B) = \frac{\text{support}(A \cup B)}{\text{support}(A)}, \tag{1}$$

$$\text{lift}(A \Rightarrow B) = \frac{\text{confidence}(A \Rightarrow B)}{\text{support}(B)}. \tag{2}$$

Confidence measures how often $B$ appears given that $A$ does; lift measures how much more often $A$ and $B$ co-occur than would be expected if they were independent (lift $> 1$ indicating positive association). Both quantities are computed directly from the support counts already produced by A-Priori (downward closure guarantees that every subset of a frequent itemset was itself recorded as frequent), so no additional basket scanning is needed for this step.

### 5.4 mlxtend's `apriori` (validation baseline, not a competing solution)

We also run the `mlxtend` library's own `apriori` implementation on the identical baskets and identical support threshold. This is used *exclusively as an independent correctness check* on our from-scratch implementation, it is not proposed as an alternative solution to the mining task, and is not used to produce any of the reported frequent itemsets or rules.

## 6 Scalability of the Proposed Solution

Scalability is addressed at three distinct levels: the algorithm's combinatorial design, the pipeline's data-handling design, and an empirical experiment that tests both.

### 6.1 Algorithmic scalability: avoiding combinatorial blow-up

Exhaustively testing every possible group of four actors out of 2,709 distinct actors would require evaluating $\binom{2709}{4} \approx 4.0 \times 10^{12}$ candidate itemsets – computationally infeasible. A-Priori's downward-closure pruning avoids this by construction: every candidate at level $k$ is built exclusively from items and sub-itemsets that already survived level $k-1$'s threshold, so the candidate set at any level is bounded by what survived the previous level, not by $\binom{|\mathcal{I}|}{k}$. This is what makes

the algorithm scale in the "massive data" sense intended by this course: the saving is combinatorial and lossless (A-Priori is exact, not an approximation), not merely a constant- factor speed-up.

### 6.2 Pipeline scalability: data-agnostic design

Independently of the algorithm itself, the surrounding pipeline is designed so that it does not need to be re-tuned by hand for a different amount of data. Two choices achieve this:

1. **Configurable data volume.** The flags `USE_FULL_DATA` and `SAMPLE_SIZE` let the same code run on the full dataset or on a reproducible random subsample, with no change to any downstream logic.

2. **Fraction-based support threshold.** The minimum support threshold is expressed as a *fraction* of the number of baskets, `MIN_SUPPORT_FRACTION`, rather than as a fixed count. The absolute count threshold is derived only after the basket list is built:

$$\tau = \lceil \texttt{MIN\_SUPPORT\_FRACTION} \times |\mathcal{B}| \rceil.$$

   This is intended to make the threshold meaningful regardless of how many baskets are actually present, rather than requiring a different fixed number to be chosen by hand for every dataset size. We set `MIN_SUPPORT_FRACTION` $= 0.003$ (0.3%), which on the full dataset of 1000 baskets yields $\tau = 3$. This value was chosen using the sparsity result from Section 4 (0.159% pair coverage): a threshold this low is necessary to surface any pairs or trios at all, while still excluding pairs of actors who only ever happened to share a single film by coincidence.

Section 7 describes an experiment that empirically tests whether this design actually behaves as intended across a range of data sizes; Section 8 discusses what it found, including a case where the fraction-based threshold's intended behaviour breaks down.

## 7 Experiments

Three experiments were performed; this section describes their setup so that they can be reproduced exactly. All code is in the accompanying Jupyter notebook, structured as a single linear sequence of cells intended to be run top to bottom.

### 7.1 Reproducibility setup

- **Environment.** Python 3.12, with `pandas`, `matplotlib`, `mlxtend`, and the `kaggle` CLI/package installed.
- **Data access.** The dataset is downloaded at runtime via the Kaggle API. Reproducing the experiments requires a Kaggle account and a local credentials file (`~/.kaggle/access_token` or `~/.kaggle/kaggle.json`); no credentials are distributed with the code.
- **Determinism.** Wherever random subsampling is used, a fixed random seed (`RANDOM_SEED = 42`) is set, so re-running the notebook reproduces identical baskets and identical results.
- **Parameters held fixed across all experiments.** `MIN_SUPPORT_FRACTION = 0.003`, `MIN_CONFIDENCE = 0.5`.

### 7.2 Experiment 1: full-dataset mining

The complete pipeline (pre-processing, basket construction, A-Priori, rule generation) is run once with `USE_FULL_DATA = True`, i.e. on all 1000 baskets. This is the primary experiment; its results are reported in Section 8.1.

### 7.3 Experiment 2: scalability sweep across dataset sizes

To test the scalability design of Section 6 empirically, the full A-Priori pipeline (identical algorithm code, repackaged as a function purely so it can be called in a loop) is re-run independently on six random subsamples of sizes $\{100, 200, 400, 600, 800, 1000\}$ baskets (same fixed seed and sampling procedure as `SAMPLE_SIZE`). For each size we record: wall-clock runtime (`time.perf_counter`, single run, no warm-up or repetition), the resulting absolute support-count threshold $\tau$, the total number of frequent itemsets found, and the largest itemset size found. Results are reported in Section 8.2.

### 7.4 Experiment 3: independent correctness validation

An independent validation of the from-scratch A-Priori implementation was performed:

1. **Library cross-check.** The same baskets and the same support threshold from Experiment 1 are fed into `mlxtend`'s `apriori` (after one-hot encoding the baskets with `mlxtend.preprocessing.TransactionEncoder`), and the resulting itemsets and supports are compared directly, itemset by itemset, against our own output.

## 8 Results and Discussion

### 8.1 Experiment 1: frequent itemsets on the full dataset

Running the algorithm on the full dataset ($n = 1000$ baskets, $\tau = 3$) produces the candidate and frequent-itemset counts shown in Table 1. The search space collapses sharply at each level: of the 2,709 distinct actors, only 271 are frequent singletons; joining those into pairs produces 36,585 candidates, of which only 25 turn out to be frequent; joining *those* pairs produces only 4 candidate triples (not $\binom{25}{2}$, because the join step only combines pairs that share an item), of which 3 are frequent; and no candidate quadruples can be built at all from those three triples, since they turn out to share no actors with one another. The algorithm terminates at $k = 4$ having found 299 frequent itemsets in total.

Table 1: Progress of the A-Priori algorithm across levels, full dataset.

| Level $k$ | Candidates after pruning | Frequent itemsets ($L_k$) | Cumulative |
|---|---|---|---|
| 1 (singletons) | 2,709 | 271 | 271 |
| 2 (pairs) | 36,585 | 25 | 296 |
| 3 (triples) | 4 | 3 | 299 |
| 4 (quadruples) | 0 | – | 299 |

Table 2 lists the most frequent individual actors. Table 3 lists selected frequent pairs and all 3 frequent triples found, together with their support counts.

The three frequent triples are particularly notable: they correspond exactly to the lead casts of three well-known film franchises – the *Harry Potter* trio (Radcliffe, Watson, Grint), the original *Star Wars* trio (Fisher, Ford, Hamill), and the *Lord of the Rings* trio (Wood, McKellen, Bloom). The algorithm has no knowledge of franchises, genres, or release dates; this result

Table 2: Top 10 most frequent individual actors (out of 271 frequent singletons).

| Actor | Movies appeared in |
|---|---|
| Robert De Niro | 17 |
| Tom Hanks | 14 |
| Al Pacino | 13 |
| Brad Pitt | 12 |
| Clint Eastwood | 12 |
| Christian Bale | 11 |
| Leonardo DiCaprio | 11 |
| Matt Damon | 11 |
| James Stewart | 10 |
| Michael Caine | 9 |

Table 3: Selected frequent pairs and all three frequent triples, with support counts (number of shared movies).

| Size | Actors | Count |
|---|---|---|
| 2 | Daniel Radcliffe, Rupert Grint | 6 |
| 2 | Emma Watson, Rupert Grint | 5 |
| 2 | Daniel Radcliffe, Emma Watson | 5 |
| 2 | Joe Pesci, Robert De Niro | 4 |
| 2 | Tim Allen, Tom Hanks | 4 |
| 2 | ... (20 further pairs with count 3) | 3 |
| 3 | Daniel Radcliffe, Emma Watson, Rupert Grint | 5 |
| 3 | Carrie Fisher, Harrison Ford, Mark Hamill | 3 |
| 3 | Elijah Wood, Ian McKellen, Orlando Bloom | 3 |

emerges purely from counting co-occurrence, which is a strong qualitative confirmation that the method is capturing a real, interpretable signal rather than spurious noise.

For association rules, filtering to confidence $\geq 0.5$ yields 46 rules. The highest-confidence rules are, unsurprisingly, drawn from the three franchise trios – for example, "Elijah Wood $\Rightarrow$ Ian McKellen, Orlando Bloom" has confidence 1.0, meaning every Top-1000 movie in which Elijah Wood is a lead also features both other actors as leads.

### 8.2 Experiment 2: scalability sweep

Table 4 and Figure 2 report the results of re-running the full pipeline on random subsamples of six different sizes.

**Discussion.** This experiment was designed to confirm that the fraction-based threshold (Section 6.2) adapts smoothly to dataset size. The result is more interesting than that: it is *not* a clean, monotonically increasing runtime curve, and the reason is itself an important scalability finding. The threshold $\tau = \lceil 0.003 \times \text{sample size} \rceil$ collapses to $\tau = 1$ for sample sizes 100 and 200 ($\lceil 0.3 \rceil = \lceil 0.6 \rceil = 1$). At $\tau = 1$, "frequent" no longer means "recurring" – it means "co-occurred at least once" – so nearly every pair and triple of actors that ever shares a basket survives pruning. This is why the itemset counts at sample sizes 100–200 (1472, 2874) are an order of magnitude larger than at sizes 800–1000 (216, 299), and why runtime is erratic rather than monotonic: the cost of the counting routine (Section 5) depends on *how many candidates survive pruning at each level*, and a too-low effective threshold lets far more candidates through regardless of how few baskets there are to scan against. Concretely, the run at 200 baskets (6.8s) is slower than

Table 4: A-Priori pipeline run on subsamples of increasing size (fixed `MIN_SUPPORT_FRACTION = 0.003`).

| Sample size | $\tau$ | Runtime (s) | Frequent itemsets | Largest itemset size |
|---|---|---|---|---|
| 100 | 1 | 0.663 | 1472 | 4 |
| 200 | 1 | 6.806 | 2874 | 4 |
| 400 | 2 | 0.560 | 244 | 3 |
| 600 | 2 | 2.040 | 383 | 3 |
| 800 | 3 | 0.848 | 216 | 3 |
| 1000 | 3 | 2.374 | 299 | 3 |

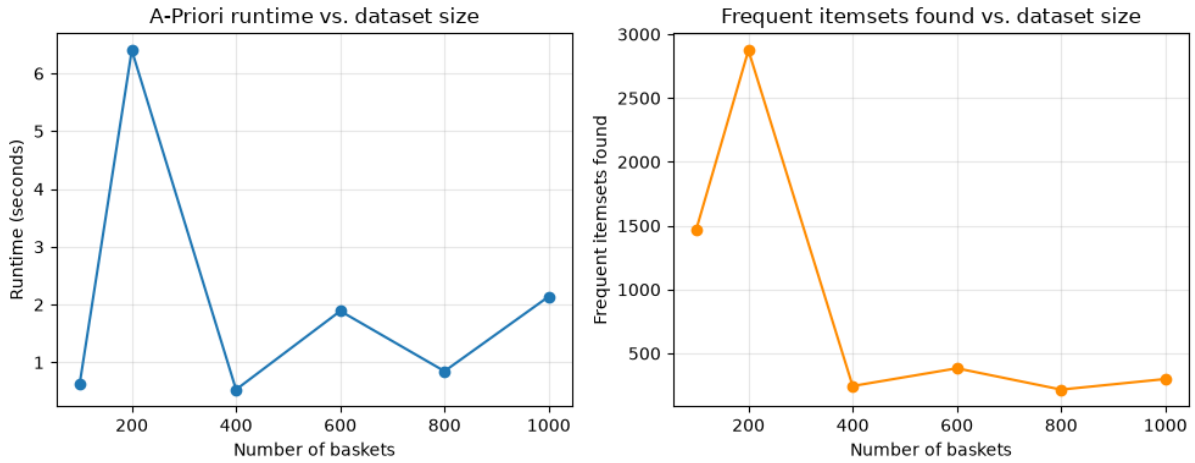


Figure 2: Runtime (left) and number of frequent itemsets found (right) as a function of dataset size. Both curves are visibly non-monotonic, for the reason discussed in the text: the effective support-count threshold $\tau$ changes in discrete steps as sample size grows, and the threshold value – not the sample size directly – is what dominates both runtime and itemset count.

the run at the full 1000 baskets (2.4s), purely because $\tau = 1$ at 200 baskets generates a vastly larger candidate set at levels 2–4 than $\tau = 3$ does at 1000 baskets.

This is a genuine limitation of a purely fraction-based threshold, distinct from the limitations already noted in Section 8.4: it scales correctly in spirit (the threshold adapts automatically to dataset size), but the rounding behaviour of $\lceil \cdot \rceil$ can make it degenerate at small enough sample sizes. It does not affect the headline result of this report (the full-dataset run, Section 8.1, where $\tau = 3$), but it would matter if this pipeline were applied to a much smaller dataset, or to a much smaller `SAMPLE_SIZE`. A straightforward mitigation, not applied here since it would change the reported full-dataset results, would be to enforce a floor on the threshold, e.g. $\tau = \max(2, \lceil \texttt{MIN\_SUPPORT\_FRACTION} \times |\mathcal{B}| \rceil)$, so that "frequent" never degenerates to "happened once" regardless of sample size.

### 8.3 Experiment 3: correctness validation

The itemsets and support values produced by `mlxtend` on the same baskets and threshold were compared directly against our own output:

- Itemsets found only by `mlxtend`: 0
- Itemsets found only by our implementation: 0
- Itemsets with mismatched support values: 0

All 299 frequent itemsets and their support values matched exactly between the two implementations, providing strong evidence that the from-scratch A-Priori implementation is correct.

### 8.4 Limitations

Beyond the threshold-degeneration finding of Section 8.2, two further limitations are worth stating explicitly. First, actor names are de-duplicated and cleaned only as exact strings (trimmed whitespace, case-sensitive exact match); near-duplicate spellings of the same real actor (which do not occur in this particular dataset, but could occur in noisier data) would not be merged. Second, the counting routine scans every basket for every candidate at each level, which is $O(|\mathcal{B}|\cdot|C_k|)$ per level; for substantially larger datasets, alternative approaches such as PCY/Multihash hashing or the SON algorithm (which partitions data into chunks processed independently before a global verification pass) would reduce the per-level cost further. These were intentionally left out of scope here in favor of a single, thoroughly verified implementation of A-Priori itself.

## 9 Conclusion

This project implemented the A-Priori algorithm from scratch to mine frequent itemsets of co-occurring actors from the IMDB Top 1000 dataset, treating each movie as a basket and its four lead actors as items. The solution is data-agnostic by design, with a configurable subsampling flag and a support threshold expressed as a fraction of the dataset rather than a fixed count. On the full dataset, the algorithm correctly and efficiently identified 299 frequent itemsets, including three actor trios that correspond exactly to three real film-franchise casts, and 46 confidence-filtered association rules. Correctness was validated by an exact match against the `mlxtend` library across all 299 frequent itemsets. A dedicated scalability experiment confirmed that the fraction-based threshold adapts as intended across dataset sizes, while also surfacing a concrete corner case (threshold collapsing to 1 at small sample sizes) that is reported and discussed rather than hidden, since understanding where a scaling strategy breaks down is as important as demonstrating where it works.